

VoxPay — Arquitectura del Sistema

21 de septiembre de 2026 · Rondnt

Resumen

VoxPay es un POS SaaS por voz: el comerciante habla, un agente de IA interpreta el cobro y Stellar lo ejecuta, reparte y deja evidencia verificable en testnet. Participa en Stellar Build Peru, track Open Build. El problema: pequeños comercios e independientes cobran con procesos manuales, comisiones y liquidaciones lentas. VoxPay permite crear un cobro diciendo, por ejemplo, “Cobra 30 USDC por el pedido 52 y separa 3 USDC de propina para José”, sin que el comerciante sepa de blockchain.

Stellar es el corazón del sistema, no un adorno:

- Las órdenes viven en un contrato Soroban, que es la fuente de verdad de montos, estados y reglas de reparto.
- El cliente paga una sola vez al contrato; el contrato valida, reparte el USDC y emite un evento.
- El agente de IA responde consultas (“¿ya pagaron el pedido 52?”) leyendo el contrato y sus eventos, no una base de datos propia.
- Sin Stellar no existen los pedidos, la liquidación, el reparto ni las respuestas del agente.

Vista general

El sistema tiene cinco componentes desplegables por separado: frontend, API, worker, servicio de IA y contrato, más PostgreSQL y Redis como infraestructura.

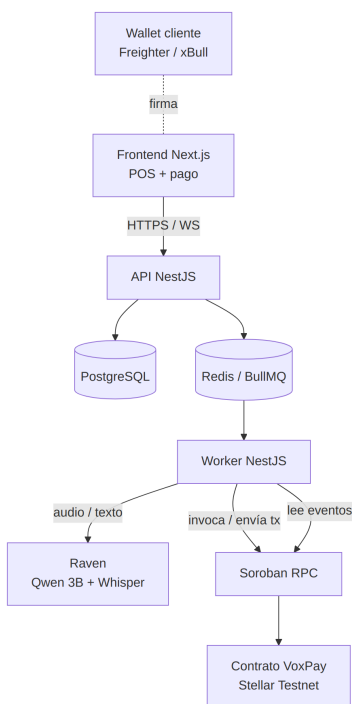


Diagrama de arquitectura general

El frontend solo muestra y firma; toda la lógica de negocio y de Stellar vive en el backend, y la verdad de los pagos vive en el contrato.

Componente	Responsabilidad
Frontend (Next.js)	Vista: POS por voz, dashboard, página pública de pago, firma con wallet
API (NestJS)	Controladores HTTP/WS, autenticación, validación, orquestación de casos de uso
Worker (NestJS)	Procesa colas: IA, envío de transacciones, indexación de eventos
Raven (Python)	Voz a texto e interpretación a JSON estructurado
Contrato Soroban	Órdenes, pagos, reparto de fondos y eventos on-chain
PostgreSQL	Tenants, usuarios, comerciantes, agenda de destinatarios, caché de órdenes
Redis	Colas BullMQ, locks, rate limiting, caché

Stack tecnológico

Todo lo que toca Stellar está en TypeScript, donde el SDK es más maduro; la IA se mantiene en Python como servicio independiente.

Capa	Tecnología	Deploy
Frontend	Next.js (App Router) + TypeScript + Tailwind, PWA	Vercel
Wallet del cliente	Stellar Wallets Kit (Freighter, xBull), solo firma	Navegador
Backend (API + worker)	NestJS + TypeScript	Railway
Validación	class-validator (DTOs) + Zod para la salida de Raven	—
ORM / base de datos	Prisma + PostgreSQL	Railway
Colas / caché	Redis + BullMQ	Railway
Tiempo real	WebSocket (Socket.IO gateway de NestJS)	—
SDK Stellar	@stellar/stellar-sdk + bindings TS del contrato	—
Agente IA	Raven: FastAPI + Qwen 3B afinado (GGUF) + llama.cpp con grammar JSON	Railway (CPU)

Capa	Tecnología	Deploy
Voz a texto	faster-whisper dentro de Raven (alternativa: Whisper de Groq)	Railway
Contrato	Rust + soroban-sdk, desplegado con stellar-cli	Stellar Testnet
Activo	USDC de testnet	—
Desarrollo local	Docker Compose (PostgreSQL, Redis, Raven)	—
Observabilidad	pino, @nestjs/terminus, Swagger	—

Estructura del repositorio

Un solo repositorio con cuatro proyectos independientes: cada uno tiene su propio package.json (o equivalente), sus dependencias, su Dockerfile y su pipeline de deploy. Frontend y backend solo se comunican por HTTP/WebSocket, nunca importándose código.

```

voxpay/
├── frontend/      # Next.js — capa de vista
├── backend/       # NestJS — controladores y modelo (API + worker)
├── ai-service/    # Raven (Python) — voz e interpretación
├── contracts/
│   └── voxpay/    # Rust / Soroban
├── docs/          # ADRs, diagramas, guía de demo
├── docker-compose.yml # PostgreSQL, Redis, Raven para desarrollo local
├── .github/workflows/ # un workflow por proyecto, filtrado por ruta
└── README.md

```

Reglas del repositorio:

- Cada carpeta se construye y despliega sola; un cambio en frontend/ no redespiega el backend.
- El contrato de API entre frontend y backend es el Swagger (OpenAPI) que genera NestJS; el frontend genera su cliente tipado desde ahí con openapi-typescript.
- Los bindings TS del contrato Soroban se generan con stellar contract bindings typescript dentro de backend/, porque solo el backend habla con el contrato.
- Variables de entorno por proyecto (.env.example en cada carpeta), nunca compartidas.

Frontend (Vista)

El frontend es la capa de Vista del MVC: muestra datos, captura voz y firma transacciones, pero no contiene reglas de negocio ni habla directo con Stellar RPC.

```
frontend/
├── src/
│   ├── app/
│   │   ├── (auth)/login/
│   │   ├── (merchant)/pos/      # POS por voz
│   │   ├── (merchant)/orders/  # historial y estado
│   │   ├── (merchant)/recipients/ # agenda de destinatarios
│   │   ├── (merchant)/dashboard/ # ventas del día
│   │   ├── pay/[orderId]/      # página pública de pago
│   │   ├── components/        # UI reutilizable
│   │   ├── features/          # voice, orders, payment
│   │   ├── services/          # cliente HTTP y socket hacia la API
│   │   ├── lib/wallet.ts       # Stellar Wallets Kit: conectar y firmar
│   │   ├── types/api.ts       # generado desde OpenAPI
│   │   └── middleware.ts      # protección de rutas del comerciante
│   ├── public/manifest.json    # PWA
│   └── .env.example            # NEXT_PUBLIC_API_URL, NEXT_PUBLIC_NETWORK
```

Pantalla	Qué hace
POS	Graba audio (MediaRecorder), lo envía a la API, muestra la confirmación y luego el QR
Pedidos	Lista órdenes con estado y link al hash en stellar.expert
Destinatarios	Alta de alias como “José” → wallet, y reglas de reparto fijas
Dashboard	Totales del día y consultas por voz
Pago público	El cliente conecta su wallet, firma la transacción que armó el backend y ve la confirmación

La página de pago no conoce el contrato: pide al backend la transacción `pay()` sin firmar (XDR), la firma con la wallet y la devuelve al backend para su envío.

Backend (Controlador y Modelo)

El backend aplica MVC por capas dentro de cada módulo de dominio: Controller → Service → Repository o Provider externo. Un mismo código arranca en dos procesos: main.ts (API HTTP/WS) y worker.ts (colas y eventos).

Capa MVC	En VoxPay	Regla
Vista	Frontend Next.js + DTOs de respuesta	Define qué datos ve el cliente
Controlador	*.controller.ts, *.gateway.ts	Recibe, valida el DTO, delega; sin lógica de negocio
Modelo	*.service.ts, *.repository.ts, esquema Prisma, contrato Soroban	Reglas de negocio y acceso a datos

```
backend/
├── src/
│   ├── main.ts           # proceso API
│   ├── worker.ts         # proceso worker
│   ├── app.module.ts
│   ├── common/           # guards, interceptors, filters, pipes
│   ├── config/           # @nestjs/config + validación de env
│   ├── infrastructure/
│   │   ├── prisma/       # PrismaService
│   │   ├── redis/        # conexión, locks
│   │   ├── queue/        # definición de colas BullMQ
│   │   ├── stellar/      # SorobanService, bindings, operator
│   │   └── agent/        # AgentProvider + RavenClient
│   └── modules/
│       ├── auth/
│       ├── tenants/
│       ├── merchants/
│       ├── recipients/
│       ├── orders/
│       ├── payments/
│       ├── voice-agent/
│       ├── analytics/
│       └── notifications/ # gateway WebSocket
├── prisma/schema.prisma
└── test/
```

Cada módulo sigue el mismo patrón:

```
orders/
├── orders.module.ts
├── orders.controller.ts
├── orders.service.ts
└── orders.repository.ts
```

```

└─ orders.processor.ts    # consumidor de cola (solo en worker)
└─ dto/
    └─ create-order.dto.ts
    └─ order-response.dto.ts

```

Módulo	Responsabilidad
auth	Login, JWT de comerciante, API keys de tenant
tenants	Alta de negocio SaaS, planes, límites
merchants	Perfil, wallet de cobro, autorización del operator en el contrato
recipients	Agenda alias → wallet y reglas de reparto fijas
orders	Crear orden, consultar estado, generar link y QR
payments	Armar XDR de pay(), recibir firma, enviar tx, registrar resultado
voice-agent	Enviar audio a Raven, validar la intención, pedir confirmación
analytics	Totales y resúmenes a partir de eventos indexados
notifications	Avisar al POS en tiempo real (orden pagada, error)

Endpoints principales:

Método	Ruta	Uso
POST	/v1/voice/commands	Sube audio; devuelve commandId
POST	/v1/voice/commands/:id/confirm	Confirma la intención y crea la orden
GET	/v1/orders/:id	Estado de la orden
GET	/v1/public/orders/:id	Datos públicos para la página de pago
POST	/v1/public/orders/:id/tx	Devuelve el XDR de pay() sin firmar
POST	/v1/public/orders/:id/submit	Recibe el XDR firmado y lo envía
GET	/v1/analytics/today	Totales del día
CRUD	/v1/recipients	Agenda de destinatarios

Servicio de IA (Raven)

Raven transforma audio en una intención estructurada; nunca firma, nunca mueve fondos y nunca decide sin confirmación del comerciante.

```
ai-service/
├── app/
│   ├── main.py          # FastAPI
│   ├── routes/          # /transcribe, /interpret, /health
│   ├── stt/             # faster-whisper
│   ├── llm/             # cliente llama.cpp + grammar JSON
│   └── schemas/         # intenciones de VoxPay
├── models/              # Qwen 3B afinado en GGUF (fuera de git)
└── Dockerfile
```

Intenciones soportadas en el MVP: create_order, get_order_status, get_sales_summary y unknown. La salida se fuerza con grammar JSON en llama.cpp, por ejemplo:

```
{
  "intent": "create_order",
  "amount": 30,
  "asset": "USDC",
  "order_ref": "52",
  "splits": [{ "recipient_alias": "José", "amount": 3, "type": "tip" }],
  "confidence": 0.93
}
```

Controles en el backend antes de mostrar la confirmación:

- Esquema Zod válido y montos positivos.
- La suma de repartos no supera el total.
- Cada alias existe en la agenda del comerciante.
- Si confidence es baja o la intención es unknown, el POS pide repetir en lugar de adivinar.

El backend depende de la interfaz AgentProvider, no de Raven directamente; cambiar de modelo o escalar Raven no toca la lógica de cobros.

Contrato Soroban

El contrato es la fuente de verdad de órdenes y pagos: registra cada orden, cobra al cliente, reparte el USDC y emite eventos verificables.

Función	Quién la invoca	Qué hace
init(admin, token)	Admin, una vez	Fija el token USDC (Stellar Asset Contract)
set_operator(merchant, operator)	Comerciante, una vez	Autoriza al backend a crear órdenes a su nombre
create_order(merchant, order_id, amount, splits)	Operator	Guarda la orden en estado Pending; valida que los

Función	Quién la invoca	Qué hace
		repartos no superen el total
pay(payer, order_id)	Cliente	Exige Pending y monto exacto, transfiere a cada destinatario y el resto al comerciante, marca Paid
cancel_order(merchant, order_id)	Comerciante u operator	Pasa a Cancelled si aún no se pagó
get_order(order_id)	Cualquiera (lectura)	Devuelve monto, repartos y estado

Estado y eventos:

- Order { merchant, amount, splits: Vec<(Address, i128)>, status, created_at }, guardado en persistent storage por order_id.
- Eventos: order_created, order_paid (con payer y monto), order_cancelled.
- Autorización con require_auth: el operator no puede crear órdenes para un comerciante que no lo autorizó, y nadie puede pagar dos veces la misma orden.

El backend nunca custodia fondos: el USDC va del cliente directamente al comerciante y sus destinatarios dentro de pay().

Flujos principales

Cobro por voz

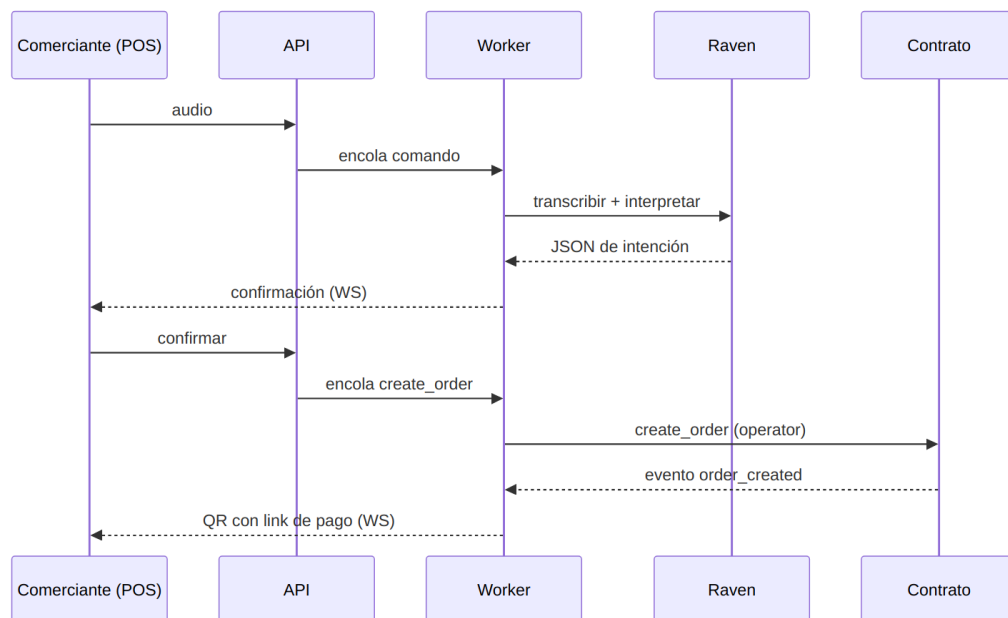


Diagrama de secuencia: cobro por voz

Pago del cliente

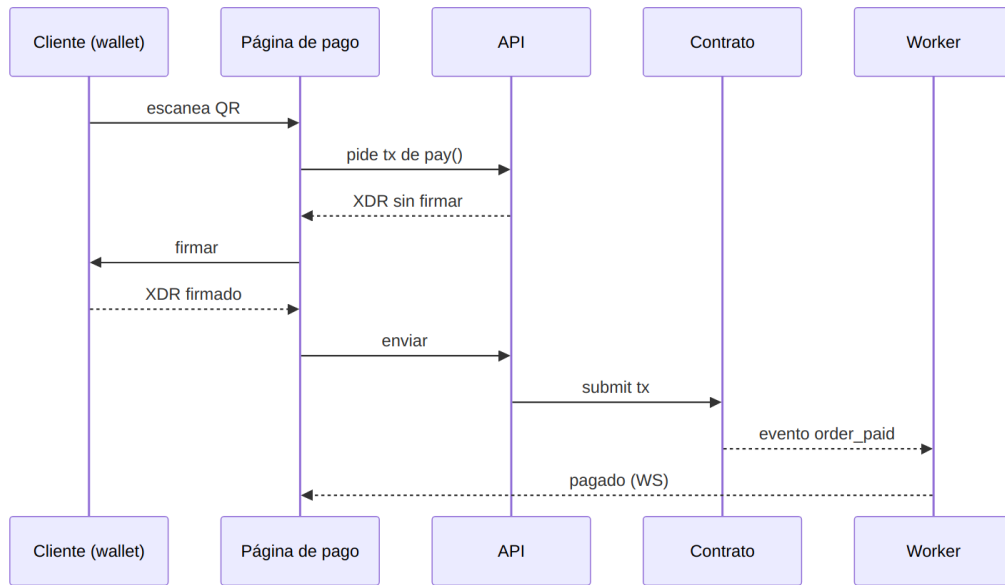


Diagrama de secuencia: pago del cliente

El POS del comerciante recibe el mismo aviso de order_paid y muestra el hash con link a stellar.expert.

Consultas por voz

- “¿Ya pagaron el pedido 52?”: Raven devuelve get_order_status; el backend llama a get_order en el contrato y responde con el estado on-chain.
- “¿Cuánto cobré hoy?”: Raven devuelve get_sales_summary; el backend suma los eventos order_paid indexados del día.

Modelo de datos

PostgreSQL guarda lo que no pertenece a la blockchain; la tabla de órdenes es una caché indexada del contrato, no la fuente de verdad.

Tabla	Campos clave	Nota
tenants	id, name, plan, api_key_hash	Un negocio SaaS
users	id, tenant_id, email, password_hash, role	Dueño o cajero
merchants	id, tenant_id, stellar_address, operator_authorized	Wallet de cobro
recipients	id, merchant_id, alias, stellar_address, default_share	“José” → wallet
voice_commands	id, merchant_id, transcript, intent_json, status	Auditoría de lo que dijo y entendió la IA

Tabla	Campos clave	Nota
orders	id, merchant_id, order_ref, amount, splits_json, status, create_tx_hash, pay_tx_hash	Espejo del contrato
chain_events	id, type, order_id, ledger, tx_hash, payload	Eventos indexados, idempotentes por tx_hash + índice
indexer_cursor	id, last_ledger	Punto de reanudación del indexador
idempotency_keys	key, tenant_id, response, expires_at	Evita órdenes duplicadas

Todas las tablas de negocio llevan tenant_id (directo o a través de merchant) para aislar datos entre clientes del SaaS.

Escalabilidad, seguridad y observabilidad

La API es sin estado y todo trabajo lento va a colas, así que cada componente escala por separado según su carga.

Componente	Cómo escala	Disparador
API	N réplicas detrás del balanceador; JWT, sin sesión en memoria	Más comerciantes conectados
Worker	N réplicas consumiendo BullMQ	Más cobros y comandos de voz
Indexador de eventos	Una sola instancia activa con lock en Redis y cursor en DB	No escala horizontalmente por diseño
Raven	Réplicas CPU; GPU si la latencia lo exige	Tiempo de respuesta de voz
WebSocket	Adaptador Redis de Socket.IO para varias réplicas	Más POS abiertos
PostgreSQL	Índices por tenant_id y order_ref; réplica de lectura para analytics	Consultas del dashboard

Multi-tenancy y seguridad:

- Un guard resuelve el tenant_id desde el JWT o la API key y lo aplica en cada repositorio.
- Rate limiting por tenant con @nestjs/throttler sobre Redis.
- Idempotency key en la confirmación de cobros y en el envío de transacciones.

- La clave secreta del operator vive solo en variables de entorno del backend (al escalar, un gestor de secretos); nunca llega al frontend.
- La IA solo propone; toda orden requiere confirmación explícita del comerciante.
- Reintentos con backoff en BullMQ para fallos de RPC o de Raven.

Observabilidad: logs estructurados con pino (con tenant_id y order_id), /health con @nestjs/terminus en API, worker y Raven, y Swagger en /docs.

Despliegue y alcance del MVP

VoxPay opera exclusivamente en Stellar Testnet en todos los entornos; no se contempla el uso de Mainnet. Escalar solo cambia réplicas y recursos, no la red ni la arquitectura.

Entorno	Frontend	Backend	IA	Red
Local	next dev	nest start (API + worker)	Docker Compose	Testnet
Demo	Vercel	Railway (API, worker, PostgreSQL, Redis)	Railway CPU + laptop de respaldo con túnel	Testnet
Escalado (futuro)	Vercel	Réplicas en Railway o contenedores	CPU o GPU según latencia	Testnet

Alcance del MVP para la demo:

- ☐ Contrato con create_order, pay, get_order y eventos, desplegado en testnet
- ☐ Voz → Raven → confirmación → orden on-chain
- ☐ QR y página de pago con firma desde wallet
- ☐ Reparto automático entre comerciante y un destinatario (propina)
- ☐ POS en tiempo real con link al hash en stellar.expert
- ☐ Consulta por voz “¿ya pagaron el pedido X?”

Posterior al MVP: path payments (pagar con XLM y recibir USDC), get_sales_summary completo, reglas de reparto persistentes, varios cajeros por comercio y facturación de planes SaaS.